

Heaps : Min-Heap y Max-Heap

Guía de implementación desde cero usando arreglos

Ing. Steven Santillan Padilla M.Sc.

Estructuras De Datos

Objetivo de la guía

Comprender qué es un **Heap**, sus propiedades fundamentales y cómo **implementar desde cero** un **Min-Heap** y un **Max-Heap** utilizando un arreglo, incluyendo las operaciones **insert**, **peek** y **extractRoot**.

Índice

1. ¿Qué es un Heap (Montículo)?	1
2. Propiedad clave: árbol binario completo	2
3. Representación en arreglo e índices	2
3.1. Diagrama conceptual (índices)	2
4. Operaciones básicas (idea y complejidad)	2
5. Heapify: restaurar la propiedad del heap	3
5.1. Heapify-Up (sift up / subir)	3
5.2. Heapify-Down (sift down / bajar)	3
6. Diagramas: Min-Heap vs Max-Heap	3
6.1. Ejemplo de Min-Heap	3
6.2. Ejemplo de Max-Heap	3
7. Implementación en Java desde cero (Min-Heap y Max-Heap)	4
7.1. Idea clave: función de prioridad	4
7.2. Código completo	4
8. Complejidad (resumen)	7
9. Errores comunes	7
10. Ejercicios recomendados	8

1. ¿Qué es un Heap (Montículo)?

Un **Heap** es una estructura de datos basada en un **árbol binario completo** (complete binary tree) que mantiene una **propiedad de prioridad**:

- **Min-Heap:** cada padre es **menor o igual** que sus hijos. El **mínimo** está en la raíz.
- **Max-Heap:** cada padre es **mayor o igual** que sus hijos. El **máximo** está en la raíz.

Nota importante

En clases a veces se menciona “grafos”, pero un heap se estudia como **árbol binario** (aunque todo árbol puede verse como un caso particular de grafo). Su implementación típica NO usa listas de adyacencia; usa un **arreglo** por eficiencia.

2. Propiedad clave: árbol binario completo

Un heap debe ser un **árbol binario completo**:

- Se llena **por niveles**, de izquierda a derecha.
- No existen “huecos” en el medio del árbol.

Esto permite guardar el heap en un **arreglo** sin desperdiciar memoria.

3. Representación en arreglo e índices

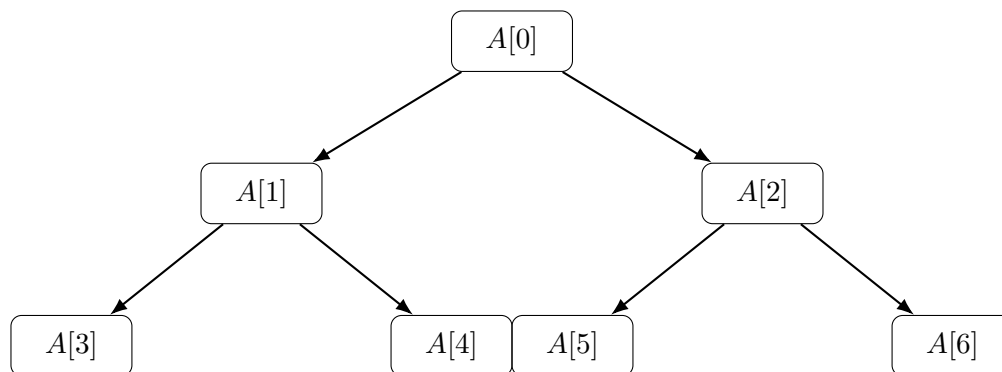
Usaremos índices desde 0 (lo más común en Java).

Relación índice-padre/hijos (índices desde 0)

Para un nodo en posición i en el arreglo:

$$\text{padre}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor, \quad \text{izq}(i) = 2i+1, \quad \text{der}(i) = 2i+2$$

3.1. Diagrama conceptual (índices)



4. Operaciones básicas (idea y complejidad)

- **peek()**: ver la raíz (mínimo en Min-Heap o máximo en Max-Heap). **O(1)**
- **insert(x)**: insertar al final (para mantener árbol completo) y luego “subir” con **heapify-up**. **O(log n)**

- **extractRoot()**: extraer raíz, mover último a la raíz y luego “bajar” con **heapify-down**. $O(\log n)$

¿Por qué $O(\log n)$?

La altura de un árbol binario completo con n nodos es $\Theta(\log n)$. En **heapify-up** y **heapify-down**, como máximo subimos o bajamos esa altura.

5. Heapify: restaurar la propiedad del heap

5.1. Heapify-Up (sift up / subir)

Se usa después de **insertar**. Si el nuevo elemento viola la propiedad, se intercambia con su padre hasta que cumpla.

- En **Min-Heap**: el elemento sube si es **menor** que su padre.
- En **Max-Heap**: el elemento sube si es **mayor** que su padre.

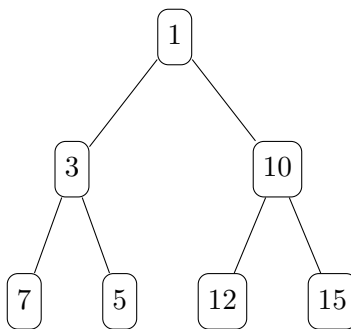
5.2. Heapify-Down (sift down / bajar)

Se usa después de **extraer** la raíz. Si el elemento en la raíz viola la propiedad, se intercambia con el hijo de **mayor prioridad**.

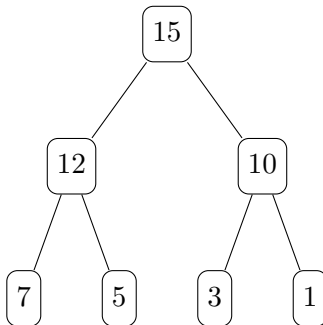
- En **Min-Heap**: se baja intercambiando con el **hijo más pequeño**.
- En **Max-Heap**: se baja intercambiando con el **hijo más grande**.

6. Diagramas: Min-Heap vs Max-Heap

6.1. Ejemplo de Min-Heap



6.2. Ejemplo de Max-Heap



7. Implementación en Java desde cero (Min-Heap y Max-Heap)

La estrategia es implementar una clase `Heap` que funcione como Min-Heap o Max-Heap usando un booleano:

- `isMinHeap = true` $\Rightarrow$ Min-Heap.
- `isMinHeap = false` $\Rightarrow$ Max-Heap.

7.1. Idea clave: función de prioridad

En lugar de duplicar código, definimos una función:

Función de prioridad

`higherPriority(a,b)` devuelve `true` si `a` debe estar arriba de `b`:

- Min-Heap: `a < b`
- Max-Heap: `a > b`

7.2. Código completo

```
1 public class Heap {
2     private int[] data;           // Arreglo que almacena el heap
3     private int size;            // Cantidad de elementos en uso
4     private boolean isMinHeap;    // true => Min-Heap, false => Max-Heap
5
6     // Constructor: capacity = capacidad inicial del arreglo
7     public Heap(int capacity, boolean isMinHeap) {
8         this.data = new int[capacity];
9         this.size = 0;
10        this.isMinHeap = isMinHeap;
11    }
12
13    // -----
14    // ndices : padre e hijos
15    // -----
16    private int parent(int i) { return (i - 1) / 2; }
17    private int left(int i)   { return 2 * i + 1; }
18    private int right(int i)  { return 2 * i + 2; }
19
20    // -----
21    // Prioridad (Min o Max)
22    // -----
23    // Devuelve true si 'a' tiene mayor prioridad que 'b'
24    // Min-Heap: menor valor tiene mayor prioridad
25    // Max-Heap: mayor valor tiene mayor prioridad
26    private boolean higherPriority(int a, int b) {
27        if (isMinHeap) {
28            return a < b;
29        } else {
30            return a > b;
31        }
32    }
33 }
```

```

33
34 // -----
35 // Intercambio (swap)
36 // -----
37 private void swap(int i, int j) {
38     int tmp = data[i];
39     data[i] = data[j];
40     data[j] = tmp;
41 }
42
43 // -----
44 // Asegurar capacidad
45 // -----
46 // Si el arreglo se llena, duplicamos el tamaño.
47 private void ensureCapacity() {
48     if (size < data.length) return;
49
50     int[] newData = new int[data.length * 2];
51     for (int i = 0; i < data.length; i++) {
52         newData[i] = data[i];
53     }
54     data = newData;
55 }
56
57 // -----
58 // peek(): ver raíz
59 // -----
60 public int peek() {
61     if (size == 0) {
62         throw new IllegalStateException("Heap vacío");
63     }
64     return data[0];
65 }
66
67 // -----
68 // insert(x): insertar
69 // -----
70 public void insert(int x) {
71     ensureCapacity();
72
73     // 1) Insertar al final para mantener árbol completo
74     data[size] = x;
75     int i = size;
76     size++;
77
78     // 2) Subir hasta restaurar propiedad
79     heapifyUp(i);
80 }
81
82 // Heapify-Up: comparar con padre y subir si corresponde
83 private void heapifyUp(int i) {
84     while (i > 0 && higherPriority(data[i], data[parent(i)])) {
85         swap(i, parent(i));
86         i = parent(i);
87     }

```

```

88     }
89
90     // -----
91     // extractRoot(): extraer ra z
92     // -----
93     public int extractRoot() {
94         if (size == 0) {
95             throw new IllegalStateException("Heap vac o");
96         }
97
98         // 1) Guardar ra z
99         int rootValue = data[0];
100
101         // 2) Poner el ltimo elemento en la ra z
102         data[0] = data[size - 1];
103         size--;
104
105         // 3) Bajar para restaurar propiedad
106         heapifyDown(0);
107
108         return rootValue;
109     }
110
111     // Heapify-Down: bajar comparando con el hijo de mayor prioridad
112     private void heapifyDown(int i) {
113         while (true) {
114             int l = left(i);
115             int r = right(i);
116
117             // Si no hay hijo izquierdo, no hay hijos
118             if (l >= size) break;
119
120             // Elegir el hijo con mayor prioridad (Min: el menor / Max: el mayor)
121             int bestChild = l;
122             if (r < size && higherPriority(data[r], data[l])) {
123                 bestChild = r;
124             }
125
126             // Si el hijo tiene m s prioridad que el padre, intercambiamos
127             if (higherPriority(data[bestChild], data[i])) {
128                 swap(i, bestChild);
129                 i = bestChild;
130             } else {
131                 // Ya cumple la propiedad
132                 break;
133             }
134         }
135     }
136
137     // -----
138     // Utilidades
139     // -----
140     public int getSize() { return size; }
141     public boolean isEmpty() { return size == 0; }

```

```

142 // Mostrar arreglo interno (solo para pruebas en clase)
143
144 public void printInternalArray() {
145     System.out.print("[");
146     for (int i = 0; i < size; i++) {
147         System.out.print(data[i]);
148         if (i < size - 1) System.out.print(", ");
149     }
150     System.out.println("]");
151 }
152
153 // -----
154 // Ejemplo de uso
155 // -----
156 public static void main(String[] args) {
157     // Min-Heap
158     Heap minHeap = new Heap(10, true);
159     minHeap.insert(7);
160     minHeap.insert(3);
161     minHeap.insert(10);
162     minHeap.insert(1);
163     minHeap.printInternalArray();
164     System.out.println("Min (peek) = " + minHeap.peek());
165     System.out.println("Extract min = " + minHeap.extractRoot());
166     minHeap.printInternalArray();
167
168     // Max-Heap
169     Heap maxHeap = new Heap(10, false);
170     maxHeap.insert(7);
171     maxHeap.insert(3);
172     maxHeap.insert(10);
173     maxHeap.insert(1);
174     maxHeap.printInternalArray();
175     System.out.println("Max (peek) = " + maxHeap.peek());
176     System.out.println("Extract max = " + maxHeap.extractRoot());
177     maxHeap.printInternalArray();
178 }
179 }

```

Listing 1: Implementación de Heap (Min-Heap/Max-Heap) usando arreglo

8. Complejidad (resumen)

Operación	Descripción	Complejidad
peek()	Ver raíz (min o max)	$O(1)$
insert(x)	Insertar y heapify-up	$O(\log n)$
extractRoot()	Extraer raíz y heapify-down	$O(\log n)$

9. Errores comunes

1. Insertar en una posición que no sea el final (rompe el árbol completo).
2. En heapifyDown, no elegir correctamente el *hijo de mayor prioridad*.

3. Confundir las fórmulas de índice: $2*i+1$, $2*i+2$, $(i-1)/2$.
4. Olvidar actualizar `size` al insertar o extraer.

10. Ejercicios recomendados

1. Implementa `buildHeap(int[] arr)` que construya el heap a partir de un arreglo.
2. Implementa `heapSort()` usando:
 - Max-Heap para ordenar ascendente.
 - Min-Heap para ordenar descendente.
3. Implementa `replaceRoot(x)`: reemplazar la raíz y reestablecer el heap.
4. Simula una **cola de prioridad**: pacientes por gravedad (Max-Heap) o tareas por tiempo (Min-Heap).

Cierre

Un heap es una estructura eficiente para manejar prioridades. La clave es mantener:

- **Árbol completo** (por eso se inserta al final).
- **Propiedad de heap** (por eso existen `heapify-up` y `heapify-down`).